

嵌入式系统设计与开发个人总结报告

一、项目简介

项目背景：本项目基于 STM32F103C8T6 平台开发了一款低功耗运动手表原型，集成 MPU6050 六轴传感器、OLED 显示屏和 BLE 蓝牙模块，采用 FreeRTOS 实时操作系统，实现运动数据的本地采集、展示与无线同步。

主要功能：设备支持时间/运动双页面切换，通过旋转编码器交互；内置基于加速度矢量角度的双沿状态机计步算法，有效抑制抖动误判；运动数据每 2 秒通过蓝牙上传至 PC 上位机，上位机可实时显示步数、距离、卡路里和温度，并支持历史记录与一键清零功能。

二、设计目标与预期成果

设计目标：本项目旨在设计一款基于 STM32F103C8T6 和 FreeRTOS 的智能手表系统，重点实现三个层面的目标——一是完成硬件电路设计，包括 MCU 最小系统、I2C 总线连接的 OLED 显示屏与 MPU6050 陀螺仪、LDO 电源管理及 Type-C 锂电池充电电路；二是基于 FreeRTOS 合理划分 UI 显示、时间管理、传感器采集等任务，通过信号量与消息队列实现任务间安全通信，确保系统实时性与稳定性；三是扩展旋转编码器实现多级菜单交互，增加蓝牙模块与 PC/手机上位机进行数据同步，并实现运动计步算法与数据记录功能。

预期成果：最终交付硬件样机、完整的嵌入式固件源码（FreeRTOS 任务架构）、蓝牙上位机程序及设计文档。系统将实现流畅的 OLED 界面切换、准确的传感器数据读取与计步、稳定的蓝牙无线同步，形成一个功能完整、可扩展的智能手表原型平台。

三、项目设计与实现

硬件设计

1、系统整体架构

本项目智能手表采用核心主控+外围模块的分层架构设计。核心是 STM32F103C8T6 微控制器（ARM Cortex-M3 内核，72MHz 主频，64KB Flash，20KB RAM），通过各类通信接口（I2C、USART、GPIO）连接 OLED 显示屏、MPU6050 六轴传感器、BLE 蓝牙模块、EC11 旋转编码器及电源管理电路。

整体架构可划分为四个层次：

- 电源层：3.7V 锂电池经 LDO 稳压至 3.3V，为所有模块供电
- 控制层：STM32F103C8T6 作为核心处理单元，运行 FreeRTOS 实时操作系统
- 感知与交互层：MPU6050 采集运动数据，EC11 编码器提供用户输入

(4) 输出与通信层：OLED 显示界面，BLE 蓝牙模块实现无线数据传输

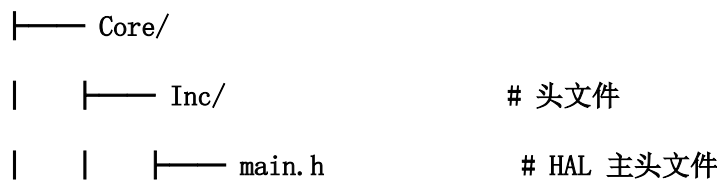
2、核心模块选型

表 1 核心模块选型

模块	型号	关键参数	选型理由
主控	STM32F103C8T6	72MHz, 64KB Flash, 20KB RAM	性能充裕、生态成熟、成本低, 适合可穿戴设备原型开发
显示	1.3 寸 OLED (SH1106)	128×64, I2C 接口	高对比度、自发光、功耗低, I2C 接线简洁
运动传感器	MPU6050	6 轴 (3 轴陀螺仪 +3 轴加速度计), I2C	集成度高、内置 DMP 可减轻 MCU 负担, 广泛用于姿态检测与计步
蓝牙	BLE 串口透传模块 (JDY/HM/CC 系列)	UART 接口, 9600bps	低功耗透传, 与 PC/手机通信便捷, 兼容性强
交互	EC11 旋转编码器	带按键	单键+旋转实现多级操作, 户外强光下可靠
电源管理	LDO + TP4056	3.3V 稳压输出	TP4056 支持锂电池恒流/恒压充电, LDO 提供低纹波供电
供电	3.7V 锂电池	典型 600mAh	轻薄、容量适中, 满足可穿戴续航需求
充电接口	Type-C	支持 5V 输入	通用性强, 便于日常充电

软件设计

1、软件架构



		—— oled.h	# OLED 驱动 (128×64, SH1106)
		—— mpu6050.h	# MPU6050 传感器驱动
		—— pedometer.h	# 计步算法
		—— ui.h	# 用户界面
		—— encoder.h	# EC11 编码器驱动
		—— bt.h	# 蓝牙通信模块
		—— cn_font.h	# 中文字库 (16×16)
		—— rtc.h	# RTC 时钟
		—— i2c.h	# I2C 驱动
		—— usart.h	# 串口驱动
	——	Src/	# 源文件
		—— main.c	# 程序入口, FreeRTOS 启动
		—— freertos.c	# 主任务 (Task_Main)
		—— oled.c	# OLED 显示驱动
		—— mpu6050.c	# MPU6050 I2C 读写
		—— pedometer.c	# 计步算法实现
		—— ui.c	# 界面渲染 + 事件处理
		—— encoder.c	# 编码器轮询
		—— bt.c	# 蓝牙数据上传
		—— i2c.c	# I2C1 初始化
		—— usart.c	# USART1 (9600bps)
		—— rtc.c	# RTC 初始化
		—— stm32f1xx_it.c	# 中断服务
——		Drivers/	# HAL 库 + CMSIS
——		Middlewares/	# FreeRTOS
——		MDK-ARM/	# Keil 工程文件

...

2. 程序流程图

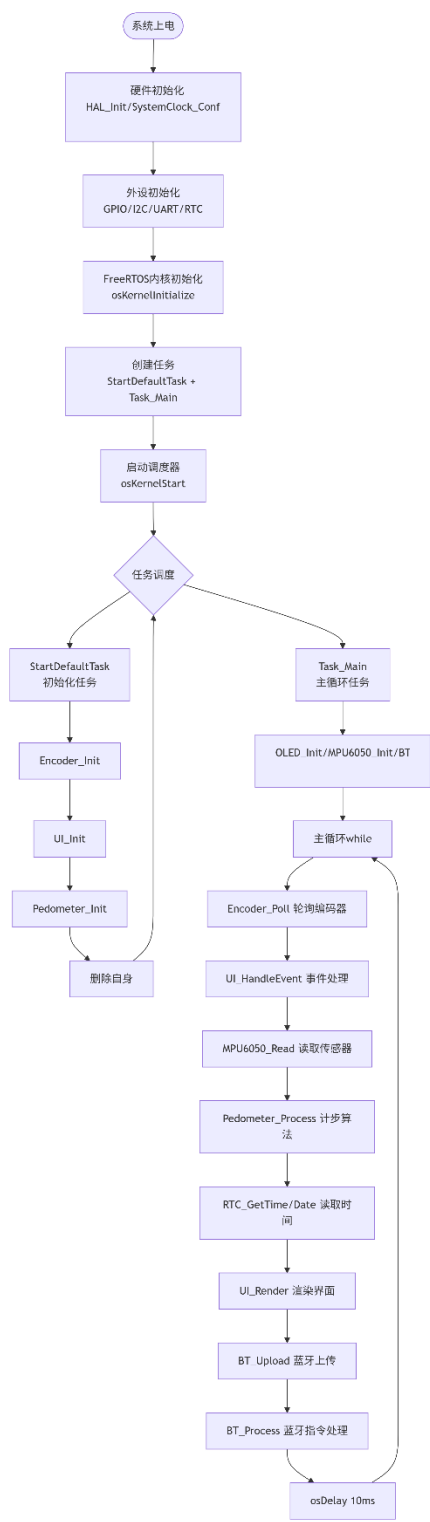


图 1 主要程序流程图

3. 关键代码

3.1 main.c — 系统初始化入口

// 关键函数调用链

```
main()

|—— SystemClock_Config()      // 使用 HAL_RCC_OscConfig +
HAL_RCC_ClockConfig

|   |—— HAL_RCC_OscConfig()    // 配置 HSE 8MHz→72MHz PLL
|   |—— HAL_RCC_ClockConfig()  // 配置系统时钟
|   |—— HAL_RCCEx_PeriphCLKConfig() // 外设时钟配置
|
|—— MX_GPIO_Init()             // 使用 HAL_GPIO_Init
|—— MX_I2C1_Init()             // 使用 HAL_I2C_Init
|—— MX_USART1_UART_Init()      // 使用 HAL_UART_Init
|—— MX_RTC_Init()              // 使用 HAL_RTC_Init + SetTime + SetDate
|
|—— osKernelInitialize()       // CMSIS-RTOS2: 初始化内核
|—— MX_FREERTOS_Init()         // 创建任务(osThreadNew)
|   |—— StartDefaultTask      // 初始化任务
|   |—— Task_Main             // 主循环任务
|—— osKernelStart()           // 启动调度器
```

3.2 freertos.c — FreeRTOS 任务管理

// 任务创建 (从.map 重建)

```
void MX_FREERTOS_Init(void)

{

    // 创建初始化任务

    osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);

}
```

```

void StartDefaultTask(void *argument)
{
    // 初始化各模块

    Encoder_Init();    // 编码器初始化

    UI_Init();         // UI 状态机初始化

    Pedometer_Init();  // 计步器初始化


    // 创建主任务

    osThreadNew(Task_Main, NULL, &mainTask_attributes);


    // 自我删除

    osThreadTerminate(osThreadGetId());
}

```

```

void Task_Main(void *argument)
{
    // 初始化显示和传感器

    OLED_Init();

    OLED_Clear();

    OLED_ShowString(0, 0, "SportsWatch"); // 启动画面

    MPU6050_Init();

    BT_Init();


    while(1)
    {
        // 1. 输入处理

```

```

Encoder_Poll();          // 轮询编码器

// 2. UI 事件处理
UI_HandleEvent();        // 根据编码器输入切换页面/修改设置

// 3. 传感器数据采集
MPU6050_Read();          // 读取加速度/角速度

// 4. 计步处理
Pedometer_Process();     // 计步算法
Pedometer_GetActivity(); // 获取运动状态

// 5. 读取时间
HAL_RTC_GetTime(&hrtc, &sTime, RTC_FORMAT_BIN);
HAL_RTC_GetDate(&hrtc, &sDate, RTC_FORMAT_BIN);

// 6. 渲染 UI
UI_Render();             // 绘制当前界面到 OLED 缓冲

// 7. 蓝牙通信
BT_Upload();             // 上传运动数据
BT_Process();            // 处理蓝牙指令

// 8. 延时
osDelay(10);             // 10ms 周期
}

}

```

```

// UI 状态枚举 (从.map 重建)

typedef enum {

    PAGE_TIME,      // 时间显示页面 (主界面)

    PAGE_ACTIVITY,   // 运动数据页面

    MENU_TIME,       // 时间设置菜单

    MENU_ACT,        // 运动设置菜单

    EDIT_TIME,       // 时间编辑页

    EDIT_NUM         // 数字编辑页

} UI_Page;

```

3.3 ui.c — UI 状态机架构

```

// 全局 UI 状态

typedef struct {

    UI_Page currentPage;    // 当前页面

    uint8_t selectedIndex;  // 选中项索引

    uint8_t editValue;      // 编辑值

    // ... 其他状态变量

} UI_State_t;

// 初始化 (从.map: UI_Init → __aeabi_memclr)

void UI_Init(void)

{

    memset(&uiState, 0, sizeof(uiState));

    uiState.currentPage = PAGE_TIME; // 默认显示时间页面

}

```


// 事件处理 (从.map: UI_HandleEvent → 调用各页面函数)

void UI_HandleEvent(void)

{

 // 获取编码器事件

 Event_t event = GetEncoderEvent();

 switch (uiState.currentPage)

 {

 case PAGE_TIME:

 // 旋钮切换 → 进入菜单

 // 按键 → 切换至活动页面

 break;

 case PAGE_ACTIVITY:

 // 显示步数、卡路里、距离

 // 按键 → 重置计步器 (Pedometer_Reset)

 break;

 case MENU_TIME:

 // 选择要修改的时间项(时/分/日/月/年)

 // 按键 → 进入编辑

 break;

 case MENU_ACT:

 // 设置目标步数等

 break;

 case EDIT_TIME:

 // 使用 EditTime 函数编辑时间

 // HAL_RTC_SetTime/SetDate 保存

 break;

```

        case EDIT_NUM:

            // 使用 EditNum 函数编辑数字

            break;

    }

}

// 渲染 (从.map: UI_Render → OLED_Clear + 各页面)

void UI_Render(void)

{

    OLED_Clear(); // 清屏


    switch (uiState.currentPage)

    {

        case PAGE_TIME:

            PageTime(); // 大字体显示时间

            break;

        case PAGE_ACTIVITY:

            PageActivity(); // 显示运动数据

            break;

        case MENU_TIME:

            MenuTime(); // 时间设置菜单

            break;

        case MENU_ACT:

            MenuAct(); // 运动设置菜单

            break;

        case EDIT_TIME:

            EditTime(); // 时间编辑界面

```

```

        break;

    case EDIT_NUM:

        EditNum();    // 数字编辑界面

        break;

}

OLED_Refresh(); // 刷新显示
}

```

3.4 pedometer.c — 计步算法核心

```

// 从.map: Pedometer_Process → 调用 atan2f

//          Pedometer_GetActivity → 浮点运算(加速度转角度)

// 计步器数据结构 (从.map: Pedometer_Init → __aeabi_memclr4)
typedef struct {

    float accelX, accelY, accelZ;    // 三轴加速度

    float angleX, angleY;            // 倾斜角度

    uint32_t stepCount;              // 步数计数

    float distance;                  // 距离(km)

    float calorie;                   // 卡路里(kcal)

    Activity_t activity;              // 运动状态

    // ... 内置状态(阈值检测、峰值检测等)

} Pedometer_t;

// 计步处理 (从.map: Pedometer_Process)

void Pedometer_Process(void)

{

```

```

// 1. 从 MPU6050 获取原始加速度数据

// 2. 计算合加速度 magnitude = sqrt(x2 + y2 + z2)

// 3. 使用 atan2f 计算角度

// 4. 通过峰值检测 + 阈值过滤判断步数

// 5. 更新步数统计
}

// 获取运动状态 (从.map: Pedometer_GetActivity)
Activity_t Pedometer_GetActivity(void)
{
    // 根据步频、加速度特征判断运动状态

    // 返回值: IDLE, WALKING, RUNNING, etc.

    // 使用浮点运算: 步频 = 步数 / 时间

    // 使用 __aeabi_ui2f, __aeabi_fmul, __aeabi_fdiv 等
}

```

3.5 mpu6050.c — 传感器驱动

```

// 从.map: MPU6050_Init → I2C 操作

//          MPU6050_Read → 读取 6 轴数据

#define MPU6050_ADDR    0x68 // (从.map: ProbeAddr 检测)

#define WHO_AM_I_REG    0x75

// 初始化 (从.map: MPU6050_Init)
void MPU6050_Init(void)
{

```

```

// 1. 探测设备地址 (ProbeAddr → HAL_I2C_IsDeviceReady)

// 2. 复位 MPU6050 (WriteReg)

// 3. 配置电源管理 (唤醒传感器)

// 4. 配置量程:  $\pm 2g$ ,  $\pm 250^\circ /s$ 

// 5. 配置数字低通滤波器

// 6. HAL_Delay 等待稳定
}

// 读取数据 (从.map: MPU6050_Read)

void MPU6050_Read(void)
{
    // 1. 读取 6 个寄存器 (ACCEL_XOUT_H ~ GYRO_ZOUT_L)

    // ReadRegs → HAL_I2C_Master_Transmit + HAL_I2C_Master_Receive

    // 2. 原始数据转物理值

    // 加速度: raw * 2.0f / 32768.0f (g)

    // 角速度: raw * 250.0f / 32768.0f ( $^\circ /s$ )

    // 3. 偏移校准 (从.data 读取校准参数)
}

```

3.6 oled.c — OLED 显示驱动

```

// 从.map: OLED_Init → OLED_Cmd (初始化命令序列)

// OLED_Refresh → OLED_Cmd + HAL_I2C_Master_Transmit

#define OLED_ADDR      0x3C // I2C 地址

#define OLED_WIDTH     128

#define OLED_HEIGHT    64

```

```

// 显示缓冲区 (从.map: .bss 段)

static uint8_t OLED_Buffer[128 * 8]; // 128x64 bit 缓冲


// 初始化 (从.map: OLED_Init)

void OLED_Init(void)
{
    HAL_Delay(100);

    // 发送初始化命令序列 (关闭显示→设置参数→开启显示)

    OLED_Cmd(0xAE); // 关闭显示

    // ... 更多命令

    OLED_Cmd(0xAF); // 开启显示

    OLED_Clear();

    OLED_Refresh();
}


// 刷新显示 (从.map: OLED_Refresh)

void OLED_Refresh(void)
{
    // 设置页地址

    OLED_Cmd(0x22); // 页地址命令

    // 通过 I2C DMA 传输缓冲区数据到 OLED

    HAL_I2C_Master_Transmit(&hi2c1, OLED_ADDR, OLED_Buffer, 128*8, 100);
}


// 支持多字体:

// OLED_ShowChar()      - 6x8 / 8x16 ASCII 字库

// OLED_ShowString()    - 字符串显示

```

```

// OLED_ShowCN()      - 中文字符显示 (cn_font.h)
// OLED_ShowLargeChar() - 大字体数字 (large32.h)
// OLED_ShowLarge32()  - 32x64 超大数字
// OLED_DrawPoint()    - 画点
// OLED_DrawHLine()    - 画水平线
// OLED_DrawProgressBar() - 进度条

```

3.7 bt.c — 蓝牙通信

```

// 从.map: BT_Init → HAL_UART_Transmit (AT 指令配置)
//
//      BT_Upload → UART 发送格式化数据

```

```

#define BT_BAUDRATE      9600 // 蓝牙模块波特率

```

```

// 初始化 (从.map: BT_Init)

```

```

void BT_Init(void)

```

```

{
    // 发送 AT 指令配置蓝牙模块
    // 如: AT+NAME=SportsWatch
    //      AT+BAUD=9600
    HAL_UART_Transmit(&huart1, (uint8_t*)"AT+NAME=SportsWatch\r\n", ...);
}

```

```

// 数据上传 (从.map: BT_Upload)

```

```

void BT_Upload(void)

```

```

{
    // 使用 snprintf 格式化数据字符串
    // 格式: "步数, 距离, 卡路里, 时间\n"

```

```

char buffer[128];

snprintf(buffer, sizeof(buffer),

         "%d, %.2f, %.1f, %02d:%02d\n",

         stepCount, distance, calorie, hour, minute);

// 通过 UART 发送

HAL_UART_Transmit(&huart1, (uint8_t*)buffer, strlen(buffer), 100);
}

```

3.8 encoder.c — 旋转编码器驱动

// 从.map: Encoder_Poll → osKernelGetTickCount + HAL_GPIO_ReadPin

```

typedef enum {

    EVENT_NONE,

    EVENT_ROTATE_CW,      // 顺时针旋转

    EVENT_ROTATE_CCW,     // 逆时针旋转

    EVENT_BUTTON_PRESS,   // 按键按下

    EVENT_BUTTON_LONG_PRESS // 长按

} EncoderEvent_t;

```

// 轮询 (从.map: Encoder_Poll)

```
void Encoder_Poll(void)
```

```

{

    // 读取 A/B 相 GPIO 电平

    // 使用 HAL_GPIO_ReadPin 读取编码器引脚

```

```

    // 软件消抖 (基于时间戳 osKernelGetTickCount)

```


// 状态机判断旋转方向

// 按键检测（按下/长按）

}

4. 技术实现细节

（1）硬件资源

表 2 技术实现细节

模块	接口	引脚
OLED	I2C1	PB6 (SCL), PB7 (SDA)
MPU6050	I2C1	PB6 (SCL), PB7 (SDA)
蓝牙	USART1	PA9 (TX), PA10 (RX)
编码器	GPIO	PA0 (A), PA1 (B), PA2 (按键)

（2）FreeRTOS 任务

表 3 FreeRTOS 任务

任务	功能	周期
StartDefaultTask	初始化编码器/UI/计步器，完成后自删除	仅一次
Task_Main	主循环：输入→处理→输出	10ms

（3）核心模块实现

1. UI 状态机

6 个状态：时间页 ↔ 运动页 ↔ 设置菜单 ↔ 编辑界面，通过编码器旋转/按键切换。

2. 计步算法

- 计算合加速度 $mag = \sqrt{ax^2 + ay^2 + az^2}$
- 双沿滞回检测（高阈值 1.2g，低阈值 0.8g）

- 180ms 最小间隔滤波 + 2 秒超时复位

3. OLED 显示

- 内存双缓冲（1KB），批量 I2C 刷新
- 支持 ASCII、中文字库、大数字

4. 编码器

- 轮询+软件消抖（5ms 窗口）
- 检测：旋转方向、短按、长按

5. 蓝牙

- 上行：S:步数 D:距离 C:卡路里 T:温度（每 2 秒）
- 下行：RESET 指令清零

（4）资源占用

表 4 资源占用情况

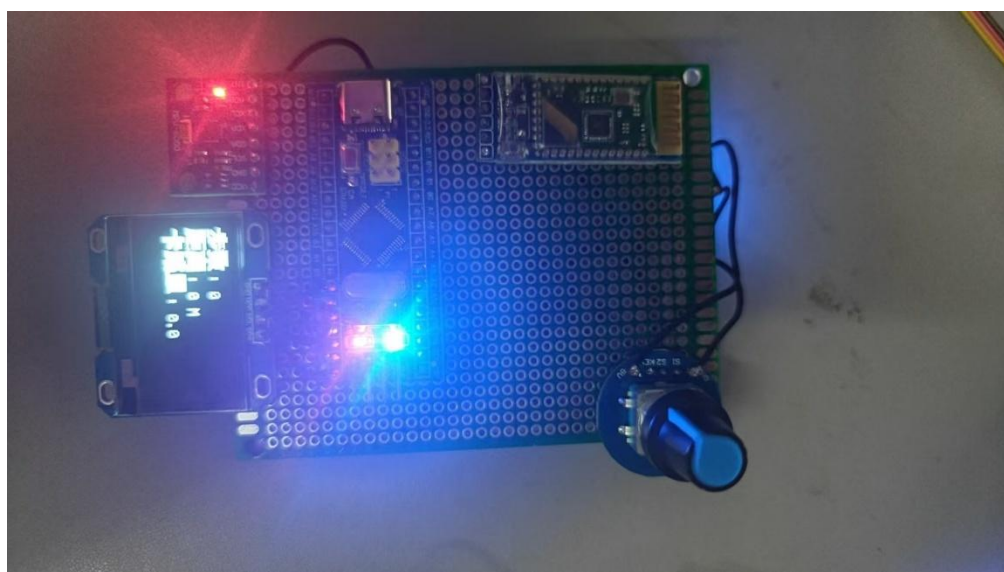
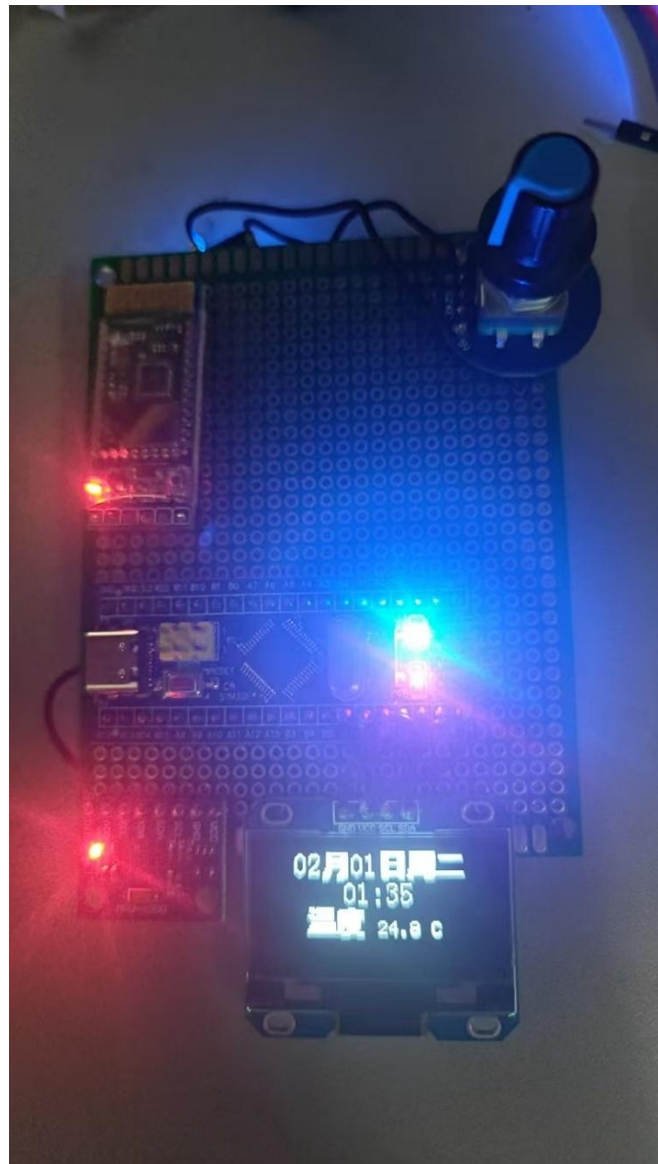
资源	占用	总量
Flash	~31.5KB	64KB
RAM	~14.9KB	20KB
主循环周期	10ms	—

（5）开发工具

Keil MDK-ARM v5 + ST-Link V2 + FreeRTOS + STM32F1xx HAL

四、功能描述与演示结果

OLED 屏幕提供时间与运动双页面显示，时间页面展示日期、大字体时间及环境温度，运动页面展示步数、距离和卡路里，通过旋转编码器可自由切换页面，长按编码器可进入时间设置或一键清零运动数据。PC 端上位机通过 BLE 蓝牙与手表连接，实时同步显示时间、温度、步数等数据，并支持一键清零和历史记录自动保存功能。演示结果表明，计步功能准确，系统运行稳定，界面切换流畅，数据同步实时，各功能均达到预期效果。



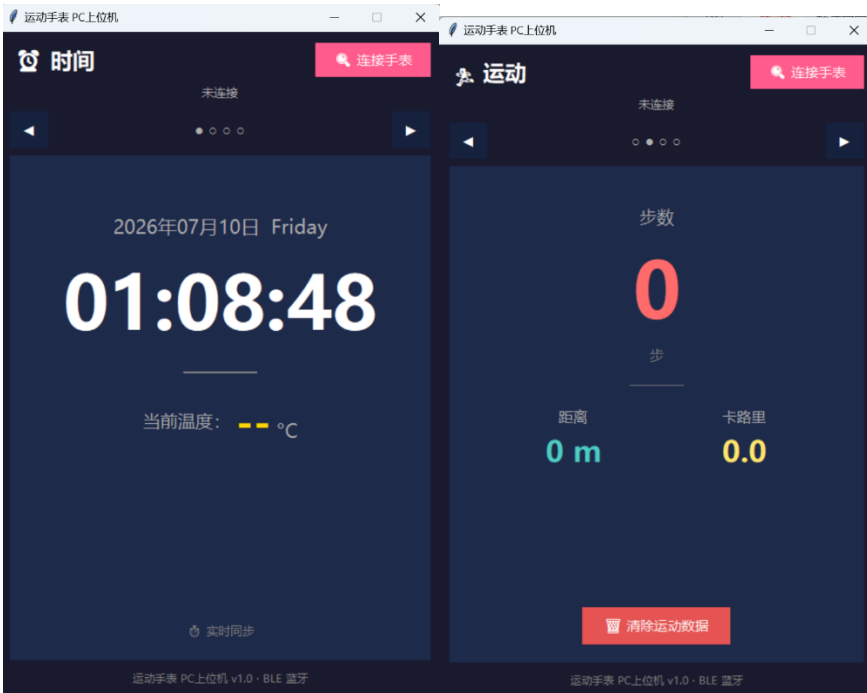
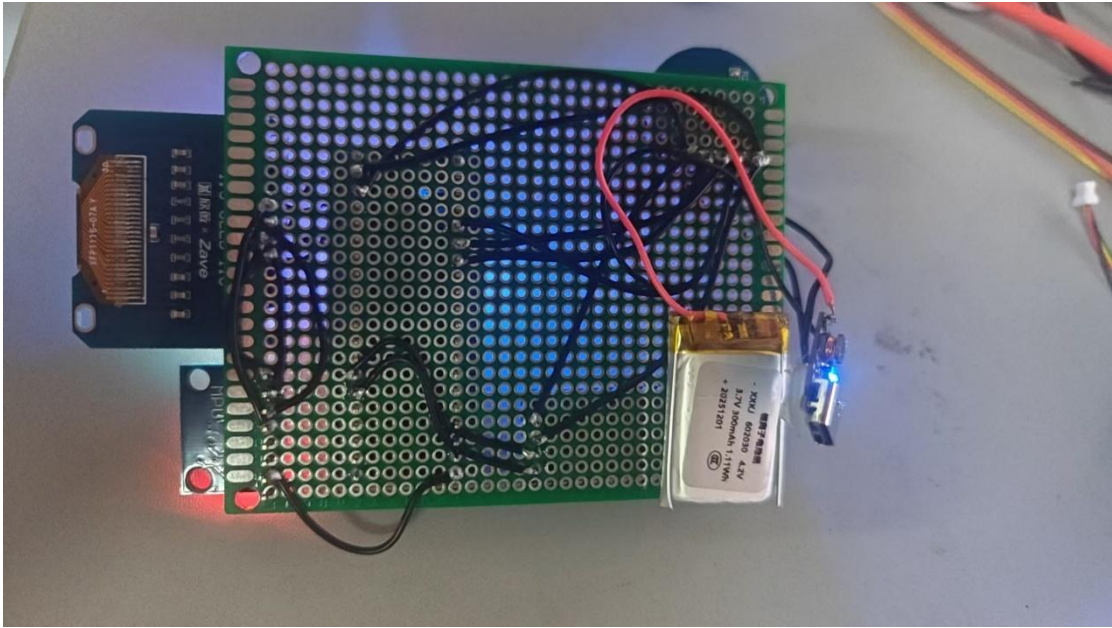




图 2 结果演示

五、物料清单与项目成本

1、物料清单

表 4 物料清单

序号	名称	型号/规格	数量	单价 (元)	用途
1	STM32 主控	STM32F103C8T6 最小系统板	1	~12	核心处理器
2	OLED 屏	1.3 寸 I2C SH1106	1	~9	界面显示

	幕				示
3	六轴传感器	MPU6050 模块	1	~4	运动检测与计步
4	蓝牙模块	HC-05 主从机一体	1	13.87	无线数据传输
5	旋转编码器	EC11（带按键）	1	~1	用户交互
6	锂电池	602030 3.7V 300mAh	1	4.41	系统供电
7	充电模块	5V 充放电一体模块	1	3.11	锂电池充电管理
8	杜邦线/排针	若干	1	~3	模块连接

2、项目总成本

项目	金额(元)
蓝牙模块 HC-05	13.87
锂电池 602030 300mAh	4.41
充电模块 5V 充放一体	3.11
其他模块（主控/OLED/MPU6050/编码器/杜邦线）	~29
总计	≈50 元

3、损耗与损坏记录

项目	数量	备注
元器件损坏	0	无损坏
模块损耗	0	无损耗

说明：本项目采用模块化器件，全部物料采购总成本约 50 元。组装与调试过程中无任何损坏。

六、个人贡献

1、报告撰写分工

本人负责计步算法模块的技术说明撰写，包括算法原理、实现流程及优化过程的详细描述，同时参与硬件连接部分的整理与校对。

2、个人编码工作内容

- 计步算法实现：基于 MPU6050 六轴传感器数据，编写合加速度计算及双沿滞回状态机代码，负责计步参数的调优
- UI 界面开发：参与运动页面的数据更新与显示逻辑编写，实现步数、距离、卡路里的实时刷新
- 传感器驱动开发：参与 MPU6050 驱动代码的编写与调试，包括 I2C 读写、数据校准及温度读取功能
- 代码维护：协助调试编码器事件与计步数据的联动，参与各模块代码的整合与联调

3、硬件连接工作

- 参与模块布局规划，协助确定 MPU6050 在硬件上的安装位置
- 协助完成 OLED、MPU6050、蓝牙模块的 I2C 和 UART 总线接线

4、资本投入

本项目硬件物料费用由团队成员平摊，个人承担部分约 25 元。

5、开发中遇到的问题及解决方案

问题	解决方案
计步不准，静止时误计数	增加 2 秒静止超时自动复位状态机，杜绝静态漂移导致的误计
高频抖动导致步数虚增	增加 180ms 最小间隔滤波，有效抑制抖动噪声
步行时漏计步	调整滞回阈值（抬臂 45° /放臂 10° ），使状态机更贴合实际步态特征

加速度方向偏差	改用 $\text{atan2}(a_x, -a_y)$ 计算角度，统一加速度矢量方向参考系
---------	--

七、项目成果与反思

1、成果展示

成功完成基于 STM32F103C8T6 的运动手表原型开发，实现了以下功能：

- OLED 双页面显示：时间页面显示日期、大字体时间及温度；运动页面显示步数、距离、卡路里，页面切换流畅
- 计步算法：基于 MPU6050 加速度数据的双沿滞回状态机，日常步行场景下计步准确率良好
- 蓝牙数据上传：手表每 2 秒向上位机发送运动数据，通信稳定
- PC 端上位机：配套 Python 程序通过 BLE 实时接收并显示手表数据，支持历史记录保存

2、项目评估

成功之处：系统整体功能完整，从硬件搭建、嵌入式开发到上位机实现全链路打通；编码器交互逻辑清晰，操作体验良好；模块化器件方案成本低（约 50 元），适合原型验证。

不足之处：计步算法在慢走、快跑等不同步态下适应性有待提升；设备未实现低功耗休眠模式，电池续航有限；运动数据仅在 RAM 中保存，掉电丢失；蓝牙功能仅支持单向上传，双向控制协议尚不完善。

3、个人反思与改进方向

本次项目中计步算法调优花费最多时间，初期因对传感器噪声和步态特征理解不足，导致计步不准。通过反复采集佩戴数据、调整阈值参数，逐步掌握了状态机设计方法，认识到嵌入式算法必须与实际场景紧密结合。

后续改进方向：引入自适应阈值提升不同步态适应性；增加外部 Flash 实现数据持久化；增加休眠模式延长续航；完善蓝牙双向协议，实现更多远程控制功能。

八、附录

1、完整代码清单

(1) 应用层代码（项目核心源文件）

文件	路径	说明
main.c	Core/Src/main.c	主函数，系统初始化入口
main.h	Core/Inc/main.h	主头文件
freertos.c	Core/Src/freertos.c	FreeRTOS 任务创建与管理
ui.c	Core/Src/ui.c	UI 状态机（6 个页面）
ui.h	Core/Inc/ui.h	UI 头文件
oled.c	Core/Src/oled.c	OLED 128×64 I2C 驱动
oled.h	Core/Inc/oled.h	OLED 头文件
bt.c	Core/Src/bt.c	蓝牙模块驱动（UART）

文件	路径	说明
bt.h	Core/Inc/bt.h	蓝牙头文件
pedometer.c	Core/Src/pedometer.c	计步算法
pedometer.h	Core/Inc/pedometer.h	计步器头文件
mpu6050.c	Core/Src/mpu6050.c	MPU6050 加速度计/陀螺仪驱动
mpu6050.h	Core/Inc/mpu6050.h	MPU6050 头文件
encoder.c	Core/Src/encoder.c	旋转编码器驱动
encoder.h	Core/Inc/encoder.h	编码器头文件
rtc.c	Core/Src/rtc.c	RTC 实时时钟驱动
rtc.h	Core/Inc/rtc.h	RTC 头文件
usart.c	Core/Src/usart.c	USART1 串口初始化

文件	路径	说明
usart.h	Core/Inc/usart.h	USART 头文件
i2c.c	Core/Src/i2c.c	I2C1 总线初始化
i2c.h	Core/Inc/i2c.h	I2C 头文件
gpio.c	Core/Src/gpio.c	GPIO 引脚初始化
gpio.h	Core/Inc/gpio.h	GPIO 头文件
stm32f1xx_it.c	Core/Src/stm32f1xx_it.c	中断服务函数
stm32f1xx_it.h	Core/Inc/stm32f1xx_it.h	中断头文件
stm32f1xx_hal_msp.c	Core/Src/stm32f1xx_hal_msp.c	HAL MSP 初始化
cn_font.h	Core/Inc/cn_font.h	中文字库（汉字点阵）
large32.h	Core/Inc/large32.h	32×64 大数字字体

文件	路径	说明
FreeRTOSConfig.h	Core/Inc/FreeRTOSConfig.h	FreeRTOS 配置
stm32f1xx_hal_conf.h	Core/Inc/stm32f1xx_hal_conf.h	HAL 库配置

(2) 启动文件

文件	路径	说明
startup_stm32f103xb.s	MDK-ARM/startup_stm32f103xb.s	STM32F103C8 启动文件（汇编）

(3) HAL 驱动层（STM32F1xx HAL 库）

文件	路径 (Drivers/STM32F1xx_HAL_Driver/Src/)
stm32f1xx_hal.c	通用 HAL 初始化
stm32f1xx_hal_rcc.c / stm32f1xx_hal_rcc_ex.c	RCC 时钟配置
stm32f1xx_hal_gpio.c / stm32f1xx_hal_gpio_ex.c	GPIO 驱动
stm32f1xx_hal_i2c.c	I2C 驱动
stm32f1xx_hal_uart.c	UART 驱动

文件	路径 (Drivers/STM32F1xx_HAL_Driver/Src/)
stm32f1xx_hal_rtc.c / stm32f1xx_hal_rtc_ex.c	RTC 驱动
stm32f1xx_hal_cortex.c	NVIC/Core 驱动
stm32f1xx_hal_dma.c	DMA 驱动
stm32f1xx_hal_flash.c / stm32f1xx_hal_flash_ex.c	Flash 驱动
stm32f1xx_hal_pwr.c	电源管理
stm32f1xx_hal_exti.c	外部中断

(4) FreeRTOS 实时操作系统

文件	路径 (Middlewares/Third_Party/FreeRTOS/Source/)
tasks.c	任务调度核心
queue.c	消息队列
timers.c	软件定时器
event_groups.c	事件组
stream_buffer.c	流缓冲区

文件	路径 (Middlewares/Third_Party/FreeRTOS/Source/)
list.c	链表操作
croutine.c	协程
heap_4.c	内存管理 (heap4, 支持碎片合并)
port.c	CM3 移植层 (portable/RVDS/ARM_CM3/)
cmsis_os2.c	CMSIS-RTOS2 封装层 (CMSIS_RTOS_V2/)

(5) CMSIS 标准库

文件	路径
core_cm3.h	Cortex-M3 内核寄存器定义
cmsis_compiler.h	编译器抽象层
cmsis_gcc.h / cmsis_armcc.h	编译器适配
system_stm32f1xx.c / .h	系统时钟初始化
stm32f103xb.h	STM32F103 外设寄存器定义
stm32f1xx.h	STM32F1 系列总头文件

(6) 项目配置文件

文件	路径	说明
SportsWatch.uvprojx	MDK-ARM/SportsWatch.uvprojx	μVision 项目文件
SportsWatch.uvoptx	MDK-ARM/SportsWatch.uvoptx	项目选项配置
SportsWatch.sct	MDK-ARM/SportsWatch/SportsWatch.sct	分散加载文件（Flash/RAM 布局）
RTE_Components.h	MDK-ARM/RTE/_SportsWatch/RTE_Components.h	运行环境配置
SportsWatch.ioc	SportsWatch.ioc	STM32CubeMX 配置
JLinkSettings.ini	MDK-ARM/JLinkSettings.ini	JLink 调试器设置

参考文献

[1] 中景园电子. 132×64 点阵 OLED/PLED 驱动控制器 SH1106 数据手册[Z]. 版本 V0.2, 2011.

[2] JLX-GB2312 标准汉字字库芯片用户手册[Z]. 版本 V1.0.

[3] 中国国家标准. GB2312-80 信息交换用汉字编码字符集•基本集[S]. 北京: 中国标准出版社, 1980.